

Manual Definitivo de Scala: Índice Secuencial (0 a Experto)

Módulo 1: The Scala Way (Fundamentos Híbridos)

Capítulo	Nombre del Capítulo	Breve Descripción
1. Configuración y Tooling Esencial	Instalación de Java JDK, uso del intérprete REPL (Read, Evaluate, Print, Loop) y configuración de <code>sbt</code> o <code>Scala CLI</code> para proyectos modernos.	
2. El Paradigma Híbrido: POO Pura y PF	Entendimiento de que todo es un objeto y toda función es un valor de primera clase . La inmutabilidad como principio central (<code>val</code> vs <code>var</code>) para sistemas concurrentes seguros.	
3. Modelado de Datos: Clases, Objetos y Traits	Uso idiomático de <code>class</code> , <code>object</code> (Singleton/Companion) y <code>trait</code> para interfaces y composición. Introducción a la potencia de <code>case class</code> para modelado de datos inmutable y eliminación de <i>boilerplate</i> .	
4. Funciones y Control de Flujo como Expresiones	Definición de funciones, funciones anónimas (lambdas) y Funciones de Orden Superior (HOFs) . Uso de <code>if-else</code> y bloques de código como expresiones que retornan un valor .	
5. Estructuras de Datos y Colecciones Funcionales	Dominio de colecciones inmutables (<code>List</code> , <code>Set</code> , <code>Map</code>) y operaciones funcionales (<code>map</code> , <code>filter</code> , <code>fold</code> , <code>reduce</code>). Diferencias clave con colecciones mutables (<code>Array</code>).	
6. Pattern Matching y ADTs Básicos	La herramienta <code>match</code> para desconstruir estructuras anidadas y complejas. Introducción a los Tipos de Datos Algebraicos (ADTs) mediante <code>sealed trait</code> o la sintaxis <code>enum</code> de Scala 3.	

Módulo 2: Scala Funcional Moderna (Tipado y Abstracciones)

Capítulo	Nombre del Capítulo	Breve Descripción
7. Manejo de Ausencia y Errores Puros	Sustitución de <code>null</code> por el tipo <code>Option[T]</code> para valores ausentes. Uso de <code>Try[T]</code> y <code>Either[E, R]</code> para manejar éxito o fracaso	

Capítulo	Nombre del Capítulo	Breve Descripción
	como valores tipados , evitando excepciones (<code>throw</code>).	
8. Abstracciones Contextuales (Scala 3: <code>given</code> y <code>using</code>)	Reemplazo del mecanismo <code>implicit</code> de Scala 2 por palabras clave enfocadas en la intención . Fundamentos de cómo <code>given</code> define valores canónicos y <code>using</code> los consume automáticamente.	
9. Type Classes y Métodos de Extensión	Implementación de patrones de Type Classes (como <code>java.util.Comparator</code> en Java) usando <code>given instances</code> . Uso de la sintaxis <code>extension</code> de Scala 3 para añadir métodos a tipos existentes.	
10. Tipos Avanzados de Scala 3: Estructura y Flexibilidad	Exploración de <code>Union Types</code> (<code>A B</code>) y <code>Intersection Types</code> (<code>A & B</code>) para flexibilidad de tipado. Uso de <code>Opaque Types</code> para crear nuevos tipos de costo cero en tiempo de ejecución.	
11. Metaprogramación y Optimización de Compilación	Uso de <code>inline</code> para forzar la expansión de código en tiempo de compilación (optimización de rendimiento). Introducción a la manipulación de ASTs (Árboles de Sintaxis Abstracta) mediante <code>Quoting</code> y <code>Splicing</code> (Macros).	

Módulo 3: El Ecosistema Industrial (Backend y Reactividad)

Capítulo	Nombre del Capítulo	Breve Descripción
12. Concurrencia: Futures y el Monad IO	Manejo de la asincronía con <code>Future</code> (la mónada que representa un valor que estará disponible en el futuro). Introducción a la Programación de Efectos Puros (IO Monad) y las Fibras (threads ligeros).	
13. Arquitectura de Efectos: Cats Effect vs ZIO	Comparativa de los dos principales <i>stacks</i> funcionales. El tipo <code>ZIO[R, E, A]</code> (Entorno, Error, Valor) y el manejo de inyección de dependencias con <code>ZLayer</code> .	

Capítulo	Nombre del Capítulo	Breve Descripción
14. Desarrollo Web Funcional	Integración de librerías HTTP modernas como <code>http4s</code> (basado en Cats Effect) y definición de APIs con Tapir para describir <i>endpoints</i> y generar documentación OpenAPI automáticamente.	
15. Modelo de Actores: Pekko Fundamentos y Patrones	Introducción al Modelo de Actores de Apache Pekko (el <i>fork</i> de Akka). Arquitectura de actores como unidades de computación aisladas que se comunican por mensajes asíncronos (<code>tell</code>).	
16. Pekko: Tolerancia a Fallos y Estado	El patrón de Supervisión (la jerarquía <i>let it crash</i>) para la tolerancia a fallos. Implementación de actores con estado (stateful vs stateless actors).	
17. Sistemas Distribuidos y Persistencia Reactiva	Uso de Cluster Sharding para distribuir entidades (actores con estado) a través de un clúster. Implementación del patrón Event Sourcing con Pekko Persistence para la recuperación de estado.	

Módulo 4: Ingeniería de Datos y Calidad (Maestría Aplicada)

Capítulo	Nombre del Capítulo	Breve Descripción
18. Apache Spark: El Dominio de Big Data	Rol de Scala como lenguaje nativo de Apache Spark . Creación de <code>SparkSession</code> y diferencia fundamental entre RDDs, DataFrames y Datasets tipados.	
19. Programación de Datos Estructurados con Spark	Manipulación de DataFrames utilizando la API de Scala: selecciones (<code>select</code>), filtrado (<code>filter</code>), agregaciones (<code>groupBy</code>) y ejecución de consultas SQL programáticas.	
20. Abstracciones de Datos: JDBC Funcional (Doobie)	Conexión a bases de datos con un enfoque funcional puro utilizando la librería Doobie (Functional JDBC layer).	
21. Diseño de Estructuras y Algoritmos Avanzados	Profundización en la inmutabilidad y estructuras de datos persistentes (<i>Homegrown Collections</i>). Análisis teórico de complejidad y notación $O(n)$.	
22. Testing de Calidad y Property-Based	Introducción a ScalaCheck para testing basado en propiedades (PBT), para validar la	

Capítulo	Nombre del Capítulo	Breve Descripción
Testing	corrección del código en lugar de solo los ejemplos.	
23. Tooling de Ingeniería y Arquitectura (sbt y Modularidad)	Configuración avanzada de <code>sbt</code> (<i>multi-project builds</i>). Uso de herramientas de refactorización y <i>linting</i> como Scalafix para imponer estándares de código.	